Exercice 1 :

1.2 : Expliquer pourquoi le code génère une erreur.

```
Sn <= A+B;          <- la erreur car celui-ci doit être déclaré après le process
Process (A, B)
begin
  Sn <= A - B;      <- on l'appel ici
end process;
```

```
Entity abc is
Port ( ____________ )
End abc
architecture arch of abc is
Begin
    Process ( ____ )
    Begin
        ____________
        ____________
        ____________
    end Process
end arch
```

Structure programme code vhdl.

justification :

→ Sn est affecté 2 fois

→ Sn est appelé en dehors du Process

→ Sn est en conflit d'affectation.

1.3 indiquer quels signaux doivent être présents dans la liste sensible d'un process pour un système combinatoire. puis pour un système séquentiel avec entrées asynchrones. puis entrée synchrone.

Système combinatoire :

→ la sortie dépend uniquement des entrées à un instant donné, sans mémoire.

Dans le process :

→ toutes les entrées utilisées qui déterminent la sortie

→ pas d'horloge.

exemple :

```
process (A, B, C)
begin
    S <= A+B+C
end Process
```

Système séquentiel avec entrée asynchrone :

→ la sortie dépend de l'historique des entrées et indépendamment de l'horloge parfois.

Dans le process :

→ l'horloge

→ le reset asynchrone, tous les signaux asynchrones

exemple :

```
Process ( clk, clear )
begin
    if ( clear = '1' ) then
        S <= (other => '0')    -- Reinitialisation asynchrone
    elsif ( clk' event and clk = '1' ) then
        S <= s + 1;
    end if
end Process
```

Système séquentiel avec entrée Synchrone
→ toutes les actions dépendent de l'horloge

Dans le process :
→ l'horloge
→ aucune entrée ne doit être mis ici

exemple :

```
Process
begin
    if rising_edge (clk) then
        Q <= D;
    end if;
end Process
```

1.4 : Expliquer la différence entre ces structure :
if then elsif et case when.

if then elsif : utilisé pour traiter des conditions générales qui peuvent être indépendantes.

Case when : utilisé pour traiter une variable sur plusieurs entrées

exemple if

```
Process (A, B)
begin
    if A = '1'
        Y <= 'c'
    elsif A = '0'
        Y <= 'D'
    else
        Y <= b
    end if;
end Process
```

exemple case

```
Process ( sel)
begin
    case sel is
    when "00" => Y <= '0'
    when "01" => Y <= '1'
    when "10" => Y <= '2'
    when others => Y <= 'a'
    end case
end Process
```

Exercice 2 : Modélisation d'un compteur 8 bits.

entrée : horloge, clear.
Sortie : S (8bit)

```
Entity Compteur_8bit is                                  de compteur_8bit
Port ( clk, clear: in std_logic
       S: Buffer std_logic_vector (7 downto 0)
End Compteur_8bit                                        de compteur_8bit
architecture arch of compteur_8bits.                     de compteur_8bit
Begin
    Process (clk, clear)
    begin
        if (clear = '1') then
            S <= (other => '0')        -- Réinitialisation asynchrone
        elsif (clk'event and clk = '1') then
            if (S = '1111 1111') then
                S <= '0000 0000'
            else
                S <= S+1               S <= S-1
            end if;
        end if;
    end process
end arch.
```

Exercice 3 : machine d'état

```
ARCHITECTURE arch OF Feux IS

signal CLKin : std_logic;
signal cpt : std_logic_vector(25 downto 0);
signal S : std_logic_vector( 2 downto 0);

BEGIN

      P0 : PROCESS (clk)
      BEGIN

      IF(clk' EVENT AND clk = '1' ) THEN

         Cpt <= cpt+1;

      END IF;

      END PROCESS;

      CLKin <= Cpt(25);

      P1 : PROCESS(CLKin)
      BEGIN

      IF(CLKin' EVENT AND CLKin = '1' ) THEN
         IF( S >= 5) THEN

         S <= (others => '0');

         ELSE

         S <= S+1;

         END IF;
      END IF;


      END PROCESS;

      P2 : PROCESS (S)
      BEGIN

      CASE S IS

      when "000" => F1 <= "100" ; F2 <= "100"  ;
      when "001" => F1 <= "001"; F2 <= "100"  ;
      when "010" => F1 <= "010" ; F2 <= "100"  ;
      when "011"=> F1 <= "100" ; F2 <= "100"  ;
      when "100" => F1 <= "100" ; F2 <= "001"  ;
      when "101" => F1 <= "100" ; F2 <= "010"  ;

      when others => F1 <= "100"; F2 <= "100"  ;

      END CASE;

      END PROCESS;

END arch;
```

L'architecture : On commence par créer 3 signaux internes :